

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

TRABAJO PRÁCTICO INTEGRADOR PROGRAMACIÓN II

**Food Store - Sistema de Gestión de Pedidos de
Comida**

Comisión 3

Xavier Vergara

Fecha de Entrega: 22/06/2026

Índice

Marco Teórico.....	3
Decisiones Técnicas y Arquitectura.....	4
Dificultades y Soluciones.....	8
Bibliografía/Webgrafía.....	9
Links.....	9

Marco Teórico

El desarrollo de este sistema se fundamenta en los pilares de la **Programación Orientada a Objetos (POO)** utilizando **Java 21** como lenguaje principal. Si bien el proyecto no implementa un motor de base de datos relacional tradicional (como MySQL o PostgreSQL mediante JDBC), se simula su comportamiento y restricciones lógicas aplicando los siguientes conceptos fundamentales:

- **Abstracción y Herencia:** Se implementó una superclase abstracta **Base** que centraliza los atributos comunes de todas las entidades (ID, estado de eliminación y fecha de creación). Las entidades específicas (**Usuario**, **Producto**, **Categoria**, **Pedido**) heredan de esta clase, reduciendo drásticamente la duplicidad de código.
- **Polimorfismo e Interfaces:** La implementación de la interfaz **Calculable** en la clase **Pedido** establece un **contrato estricto** para el cálculo de los totales, permitiendo que el sistema ejecute métodos dinámicos garantizando la escalabilidad. Asimismo, la sobrescritura del método **toString()** facilitó la representación visual de los objetos en la consola.
- **Persistencia en Memoria (Colecciones):** Ante la ausencia de una base de datos relacional, la persistencia se gestionó mediante la interfaz **List** y su implementación **ArrayList**. Estas estructuras de datos dinámicas actúan como las "tablas" del sistema, albergando los objetos instanciados y permitiendo operaciones CRUD (Create, Read, Update, Delete) iterando sobre los elementos en tiempo de ejecución.
- **Encapsulamiento:** Se protegió la integridad de los datos mediante el uso de modificadores de acceso (**private**) y la exposición controlada a través de métodos **Getters** y **Setters**, delegando las validaciones numéricas y de estado a la capa de servicios.

Decisiones Técnicas y Arquitectura

Para evitar el acoplamiento y asegurar que el código sea mantenible, se optó por una **Arquitectura en Capas** estructurada por paquetes. Esta decisión permite separar claramente las responsabilidades del sistema:

1. **Capa de Dominio (`entities`, `enums`, `interfaces`):** Contiene exclusivamente las clases que modelan el negocio. No tienen conocimiento del menú ni de cómo se guardan los datos.
2. **Capa de Lógica de Negocio (`services`):** Aquí residen los "cerebros" del sistema (`CategoriaService`, `ProductoService`, etc.). Estos servicios encapsulan las listas en memoria y son los únicos autorizados para modificarlas, aplicando reglas estrictas (por ejemplo, validar la unicidad de un correo electrónico o descontar stock).
3. **Capa de Presentación (`vergara_xavier_tpi.Main`):** Se encarga únicamente de interactuar con el usuario mediante la consola (`Scanner`), capturar las entradas y delegar el procesamiento a los servicios.
4. **Gestión de Errores (`exception`):** Se crearon excepciones personalizadas (`EntidadNoEncontradaException` y `ReglaNegocioException`). Esto nos permitió abandonar el uso de validaciones engorrosas con `if-else` en el Main, reemplazándolas por bloques `try-catch` limpios que capturan y reportan los fallos lógicos sin interrumpir la ejecución del programa.

Baja Lógica (Soft Delete): Otra decisión técnica crucial fue no utilizar el método `.remove()` de las listas al eliminar un registro. En su lugar, se implementó un atributo booleano `eliminado` en la clase `Base`. Al "eliminar" un dato, este cambia a `true`, ocultándose en los listados pero manteniendo la integridad referencial histórica (por ejemplo, evitar que un pedido antiguo pierda la referencia al usuario que lo realizó).

The screenshot shows an IDE with a project explorer on the left, a source code editor in the center, and an output console on the right. The project explorer shows a project named 'VERGARA_XAVIER_TPI' with a 'Main.java' file selected. The source code editor displays the following Java code:

```
208 } catch (NumberFormatException e) {
209 }
210 break;
211 }
212 } catch (NumberFormatException e) {
213 System.out.println("Error al ingresar el número de opción.");
214 }
215 } while (opcion != 0);
216 }
217
218 // --- SUBMENÚ DE USUARIOS ---
219 private static void menuUsuarios() {
220 int opcion = -1;
221 do {
222 System.out.println("Submenú de Usuarios");
223 System.out.println("1. Listar");
224 System.out.println("2. Crear");
225 System.out.println("3. Editar");
226 System.out.println("4. Eliminar");
227 System.out.println("0. Volver al Menú Principal");
228 System.out.println("Seleccione una opción: ");
229
230 try {
231 opcion = Integer.parseInt(scanner.nextLine());
232 switch (opcion) {
233 case 1:
234 usuarios();
235 break;
236 case 2:
237 System.out.println("Ingrese nombre de usuario:");
238 String nombre = scanner.nextLine();
239 System.out.println("Ingrese descripción:");
240 String descripcion = scanner.nextLine();
241 System.out.println("Usuario creado con éxito.");
242 break;
243 case 3:
244 System.out.println("Ingrese ID de usuario a editar:");
245 int id = Integer.parseInt(scanner.nextLine());
246 System.out.println("Ingrese nueva descripción:");
247 String nuevaDescripcion = scanner.nextLine();
248 System.out.println("Usuario actualizado con éxito.");
249 break;
250 case 4:
251 System.out.println("Ingrese ID de usuario a eliminar:");
252 int idEliminar = Integer.parseInt(scanner.nextLine());
253 System.out.println("Usuario eliminado con éxito.");
254 break;
255 default:
256 System.out.println("Opción no válida.");
257 }
258 } catch (NumberFormatException e) {
259 System.out.println("Error al ingresar el número de opción.");
260 }
261 } while (opcion != 0);
262 }
```

The output console shows the following text:

```
run:
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione una opción: 1

--- GESTIÓN DE CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al Menú Principal
Seleccione: 2
Ingrese nombre de la categoría: Postres
Ingrese descripción: Comestibles variados de tipo dulce
Categoría creada con éxito!

--- GESTIÓN DE CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al Menú Principal
Seleccione: 1

--- LISTADO DE CATEGORÍAS ---
ID: 1 | Nombre: Postres | Descripción: Comestibles variados de tipo dulce

--- GESTIÓN DE CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al Menú Principal
Seleccione: |
```

Sistema en ejecución: Creación y listado de categoría.

The screenshot shows an IDE with three main panels. The left panel displays a project tree with folders like 'Source Packages', 'Test Packages', and 'Libraries'. The middle panel shows the source code of 'Main.java' with line numbers from 213 to 263. The right panel shows the 'Output - VERGARA_XAVIER_TPI (run)' window with the following text:

```
0
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione una opción: 2

--- GESTIÓN DE PRODUCTOS ---
1. Listar
2. Crear
3. Editar (Precio, Stock, Categoría)
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Cheese Burger
Precio: 12.99
Descripción: Hamburguesa con queso 120g
Stock inicial: 9
URL Imagen (o vacío): ./img.png
¿Está disponible? (true/false): true
ID de la Categoría asociada: 1
¿Producto creado con éxito!

--- GESTIÓN DE PRODUCTOS ---
1. Listar
2. Crear
3. Editar (Precio, Stock, Categoría)
4. Eliminar
0. Volver
Seleccione: 1

--- LISTADO DE PRODUCTOS ---
ID: 2 | Nombre: Cheese Burger | Precio: $12.99 | Stock: 9 | Categoría: Pa...
```

Sistema en ejecución: Creación y listado de producto.

The screenshot shows an IDE with the following components:

- Projects:** A list of projects on the left, including 'PrimerObjeto', 'relacion-1A1-BIDIRECCION', 'TrabajoPractico_05', 'TrabajoPractico_06', 'TrabajoPractico_07', 'TrabajoPractico_08', 'VERGARA_XAVIER_PARCIA', 'VERGARA_XAVIER_PARCIA', 'Vergara_Xavier_POO_S4', 'Vergara_Xavier_TP_POO', and 'VERGARA_XAVIER_TPI [ma]'. The 'VERGARA_XAVIER_TPI [ma]' project is selected, showing its source packages and libraries.
- Main.java:** The source code file is open, showing a Java program. The code includes a menu system for a food store, with options for listing, creating, editing, and deleting products. It also includes a switch statement for handling user input.
- Output - VERGARA_XAVIER_TPI (run):** The output window shows the program's execution. It displays the menu options and the user's input. The program outputs the details of a product (Hot Dog) and then displays an error message: 'Error: El stock no puede ser negativo.'

```
0
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione una opción: 2

--- GESTIÓN DE PRODUCTOS ---
1. Listar
2. Crear
3. Editar (Precio, Stock, Categoría)
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Hot Dog
Precio: 9.99
Descripción: Pancho clasico
Stock inicial: -5
URL Imagen (o vacío):
¿Está disponible? (true/false): true
ID de la Categoría asociada: 2
Error: El stock no puede ser negativo.

--- GESTIÓN DE PRODUCTOS ---
1. Listar
2. Crear
3. Editar (Precio, Stock, Categoría)
4. Eliminar
0. Volver
Seleccione: |
```

Sistema en ejecución: atrapamos error de stock negativo.

Dificultades y Soluciones

Durante el ciclo de desarrollo, surgieron diversos desafíos técnicos propios de la gestión de memoria y el diseño de la lógica de negocio, los cuales fueron resueltos iterativamente:

- **Dificultad 1: Estados iniciales y NullPointerExceptions al heredar.** Al principio, los objetos recién creados presentaban problemas al intentar filtrar aquellos eliminados o consultar su fecha de alta, ya que los atributos de la superclase `Base` no se inicializaban correctamente. **Solución:** Se reestructuró el constructor de la clase `Base` para asignar valores por defecto en el momento de la instanciación (`this.eliminado = false` y `this.createdAt = LocalDateTime.now()`), garantizando que toda entidad nazca activa y con un time-stamp válido.
- **Dificultad 2: Pérdida de inventario ante pedidos fallidos.** El escenario más complejo fue la creación de `Pedidos`. Si un usuario agregaba tres productos exitosamente, descontando el stock, pero fallaba en el cuarto (por falta de unidades), el sistema lanzaba una excepción pero el inventario de los primeros tres productos ya había sido descontado, generando inconsistencias. **Solución:** Se implementó un enfoque transaccional en el `PedidoService`. El pedido se construye temporalmente en memoria paso a paso. Si ocurre un error, un bloque `catch` invoca un método de rescate (`cancelarPedidoYRevertirStock`) que recorre los detalles ingresados y reintegra las cantidades exactas al stock de cada producto antes de descartar el pedido.
- **Dificultad 3: El salto de línea fantasma en consola.** Al interactuar con el menú, utilizar `scanner.nextInt()` generaba un error recurrente donde el programa "saltaba" la siguiente pregunta de texto al dejar el carácter de salto de línea en el buffer. **Solución:** Se estandarizó la captura de datos leyendo todo como cadenas de texto con `scanner.nextLine()` para limpiar el buffer, y posteriormente aplicando envoltorios (Wrappers) como `Integer.parseInt()` o `Double.parseDouble()` combinados con excepciones `NumberFormatException` para asegurar la robustez de las entradas.

Bibliografía/Webgrafía

Material provisto por la cátedra.

Playlists YouTube

Charly Cimino - Programación desde menos 10 en Java.

Charly Cimino - Objetos desde menos 10 en Java.

Links:

Repositorio en GitHub:

https://github.com/xaviervergara/Vergara_Xavier_TPI-Programacion-II.git

Video YouTube:

<https://youtu.be/8wXQdcj-8t4>